

Xal-Tor-Ka — panoramica e valore

Un **unico punto d'ingresso** che autentica, instrada e ospita. Xal-Tor-Ka sta davanti ai tuoi servizi: verifica *chi* entra, decide *dove* va, mette il TLS, e — se vuoi — ospita direttamente i siti in container isolati. Tutto da un pannello solo.

Documento per chi valuta l'adozione. Per il “come si fa” operativo vedi xaltorka-howto.md.
Screenshot da un'installazione dimostrativa.

In una frase

Xal-Tor-Ka è un **gatekeeper di autenticazione + reverse-proxy manager + piattaforma di hosting** in un solo binario Go, esposto come unico punto pubblico (dietro NGINX). Metti i tuoi servizi *dietro* di lui e ottieni: login centralizzato, autorizzazione per-servizio, certificati TLS automatici e — opzionale — hosting di siti in Docker isolati, gestiti dall'interfaccia.

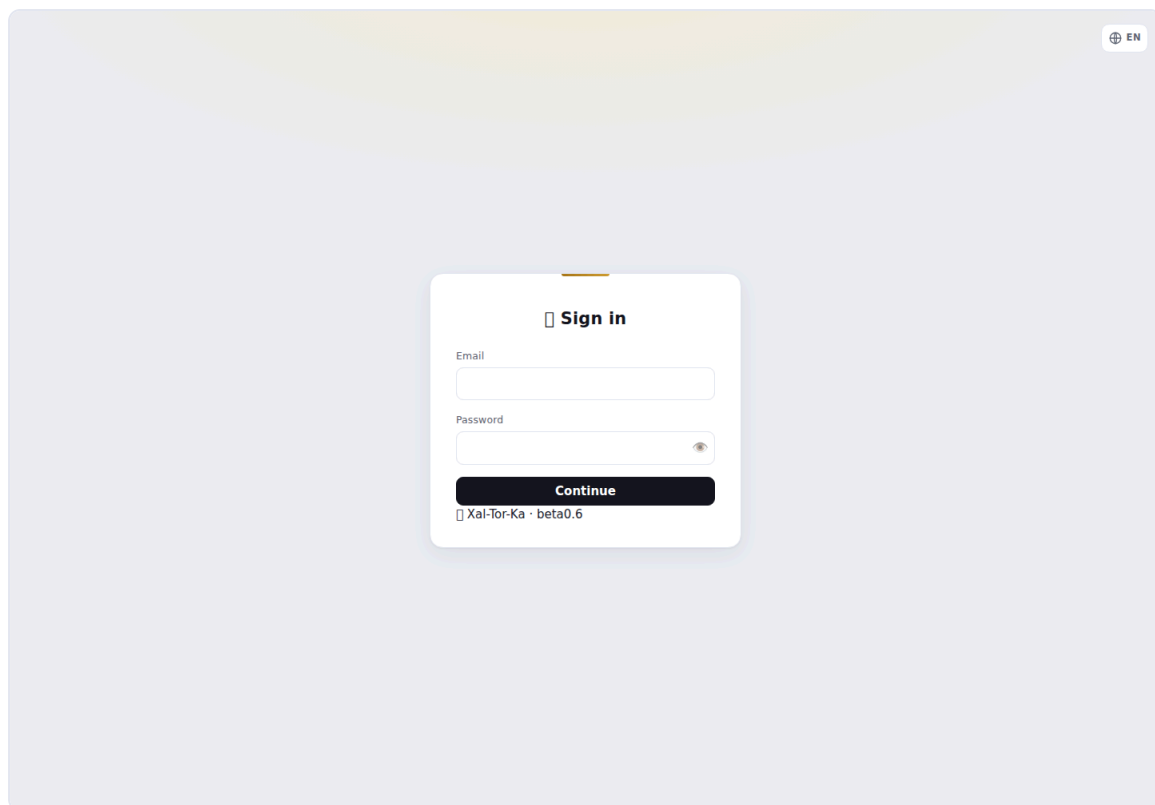
Il problema che risolve

Chi gestisce più servizi web (interni, vetrine, gestionali, un Ollama, un pannello...) si ritrova a ripetere ovunque le stesse cose: autenticazione, HTTPS, chi-può-vedere-cosa, e la fatica di mettere online un sito nuovo. Xal-Tor-Ka **centralizza tutto questo davanti**: un servizio dietro il gate non deve più preoccuparsi di login né di certificati — li mette il gate, in modo uniforme e auditabile.

Le capacità, in concreto

1. Un solo ingresso, autenticazione centralizzata e a più fonti

Login unico per tutti i servizi dietro il gate. Le **fonti d'identità sono componibili**: utenti locali con 2FA (TOTP), provider **OIDC** (Google, Microsoft, o qualsiasi issuer standard), e **LDAP / Active Directory** (bind al domain controller). Un servizio protetto non vede mai una richiesta non autenticata.



Login

2. Reverse-proxy manager con regole per servizio

Ogni servizio è un *backend* con una **regola di accesso**: `public` (aperto), `authenticated` (serve il login), `whitelist` (solo IP consentiti). Li gestisci dal pannello — host pubblico, upstream interno, regola — senza toccare file di configurazione a mano. Il catalogo dei servizi è anche la **home page** per l'utente autenticato.

Available services

Demo

demo.localhost

PROXY

myrules

myrules.localhost

PROXY

example-com/httpdocs
(hosting)

example.com

PROXY

example-com/app
(hosting)

app.example.com

PROXY

Demo service

demo.example.com

PROXY

Documentazione

EXTERNAL

Link esterno di esempio (visibile a tutti)

Xal-Tor-Ka

ADMINISTRATION

beta9.6

Services

Docker

Providers

Users

Monitoring

TLS

Hosting

Dashboard

EN

Reverse-proxy services

Services from config.json are read-only (infrastructure). Those added here live in services.json.

SERVICE	HOST / UPSTREAM	RULE	IP ALLOW				
Demo	demo.localhost http://demo:80	PUBLIC		edit	Manage SSL	disable	delete
myrules-builder-backend-1	myrules-builder-backend-1.localhost http://host.docker.internal:8765	PUBLIC		edit	Manage SSL	enable	delete
myrules	myrules.localhost http://host.docker.internal:5173	AUTHENTICATED		edit	Manage SSL	disable	delete
example-com/httpdocs (hosting)	example.com http://example-com.site:8080	PUBLIC		edit	Manage SSL	disable	delete
example-com/app (hosting)	app.example.com http://example-com-app.site:8080	PUBLIC		edit	Manage SSL	disable	delete
Demo service	demo.example.com http://demo:80	AUTHENTICATED		edit	Manage SSL	disable	delete

Add proxied service

id

name

host

path

rule

upstream

public url

www

IP allow (CIDR, opt.)

add

public: open to everyone. authenticated: users with a valid session (2FA). whitelist: only explicitly authorized users (implies authentication); administrators always have access (override).

External links

Tiles in the dashboard, not proxied.

NAME	URL	VISIBILITY		
Documentazione	https://example.com/docs	PUBLIC	disable	delete

Add link

id

name

public url

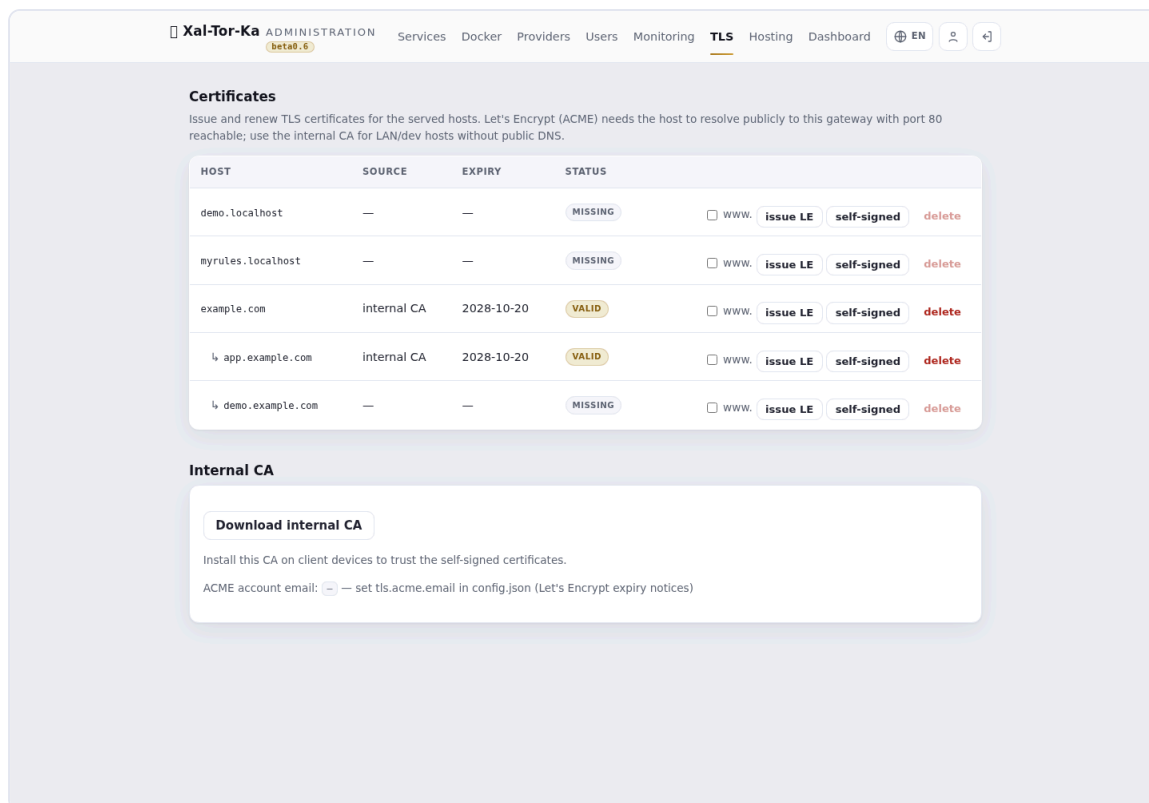
description

add

public

3. TLS ovunque, senza sbatti

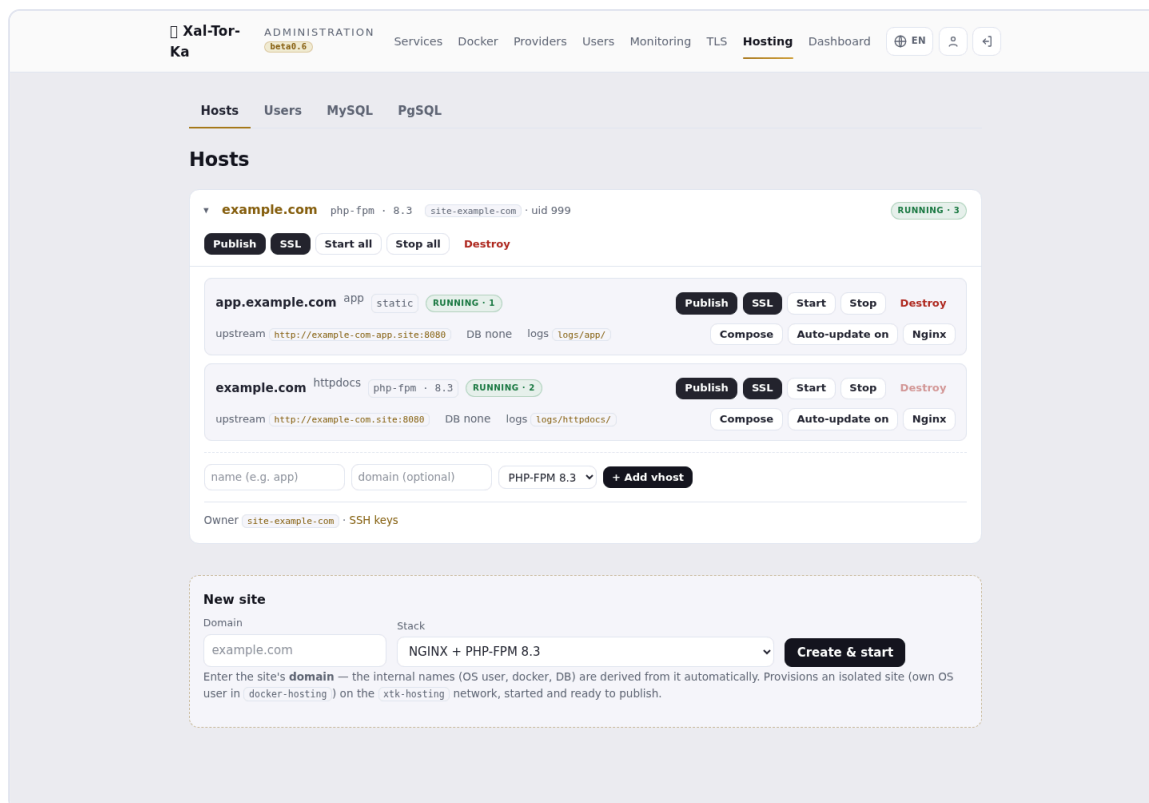
Certificati **Let's Encrypt** automatici (ACME HTTP-01) per gli host pubblici, e una **CA interna** scaricabile per i servizi LAN/dev senza DNS pubblico. I certificati seguono i servizi: pubblici un host, poi emetti il certificato con un clic. I **sottodomini** compaiono annidati sotto il dominio padre, per una gestione ordinata.



Certificati TLS

4. Piattaforma di hosting (opzionale) — siti in Docker isolati

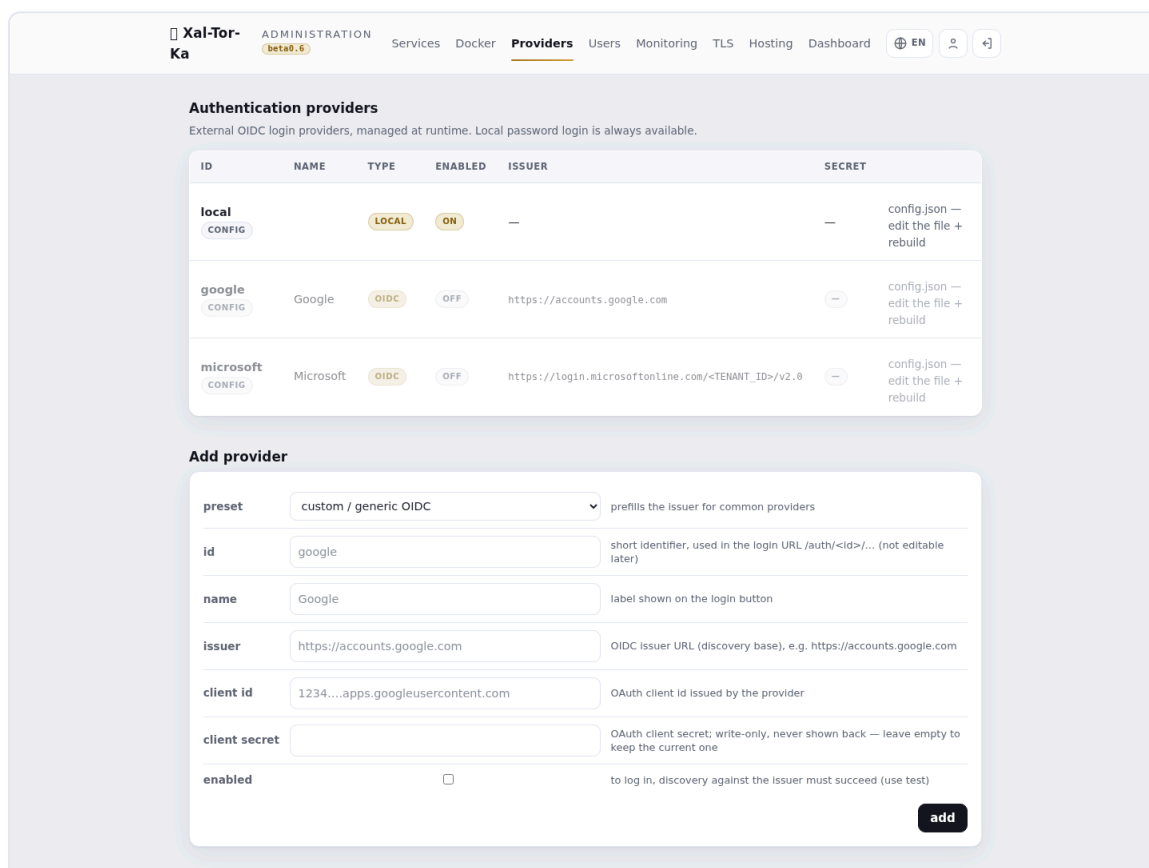
Oltre a proxare servizi esistenti, Xal-Tor-Ka sa **creare siti da zero**: ogni sito è un **utente di sistema isolato** con uno o più **vhost**, ognuno nella sua **Docker** (NGINX + PHP-FPM 8.1/8.2/8.3, Apache, statico, Node, o compose custom). Database **condivisi** MySQL/PgSQL con utenti isolati, accesso **SFTP/SSH** in chroot con chiavi, editor di compose e nginx. Dal dominio si derivano automaticamente utente, container e DB.



Piattaforma hosting

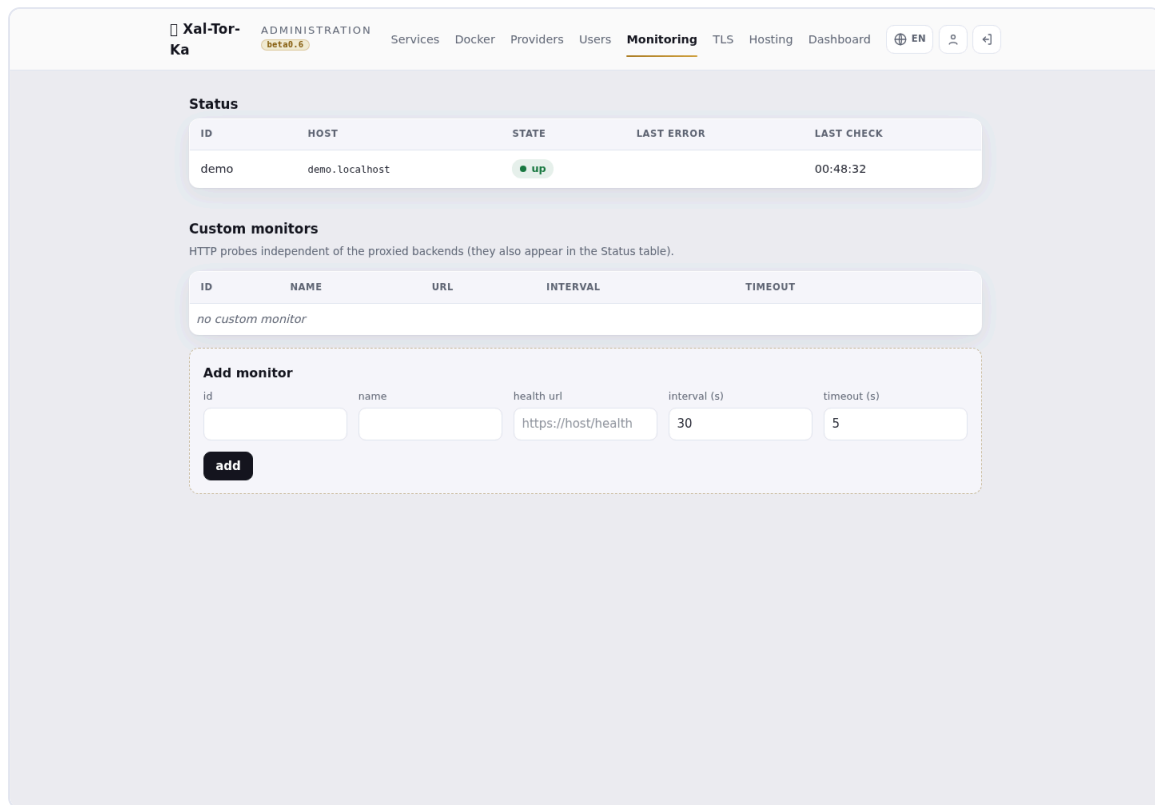
5. Identità e provider, dal pannello

Provider OIDC e utenti si aggiungono e si gestiscono runtime, senza riavvii.



6. Operatività: monitoraggio, audit, controllo remoto

Health dei backend in tempo reale, log di audit degli accessi, **whitelist IP** per l'area di amministrazione, e **notifiche/controllo remoto** opzionali via Telegram/email (log di sistema a distanza, con comandi vettati).



Monitoraggio

7. Difesa in profondità

Oltre all'auth centralizzata, strati di difesa attivabili per servizio o per host: - **Auth per-path**: proteggi singoli file/cartelle (es. `wp-login.php`, `/wp-admin/`) lasciando il resto pubblico — i bot non arrivano nemmeno alla login. - **fail2ban al firewall**: gli IP che insistono coi fallimenti auth vengono bannati (in *prerouting* nftables, efficace anche col gate in container); IP admin e LAN in whitelist. Gestione dal pannello. - **Aggiornamenti OS**: check read-only e applicazione admin-gated dei pacchetti dell'host, dal pannello. - **WAF** (in arrivo): ModSecurity + OWASP CRS davanti ai servizi, con toggle per-sito.

Il modello di sicurezza (perché ci si può fidare)

- **Un solo punto esposto** (NGINX → gate Go interno). I servizi stanno dietro, non sulla rete pubblica.
- **Area admin blindata** da whitelist IP + sessione; ogni azione è auditata.
- **Agente host vettato**: le operazioni privilegiate (creare utenti-OS, avviare Docker) passano per un agente con un **insieme fisso di script root-owned non iniettabili** — i parametri arbitrari

arrivano come variabili d'ambiente, mai come shell. La sicurezza viene dal *modello stretto*, non dal sandboxing.

- **Repo pubblico, segreti fuori:** nessuna credenziale nel codice.

Perché adottarlo

- **Un login e un HTTPS per tutto**, invece di reimplementarli in ogni servizio.
- **Autorizzazione uniforme** (public / autenticato / whitelist) decisa in un posto solo.
- **Metti online un sito** in minuti, isolato e già pronto a pubblicare — senza Plesk/cPanel.
- **Fonti d'identità aziendali** (AD/LDAP, OIDC) senza scrivere codice.
- **Auditabile e riproducibile:** ogni operazione tracciata, configurazione dichiarativa.

Casi d'uso tipici

- Mettere **autenticazione + HTTPS davanti a un servizio “nudo”** (es. un Ollama, un pannello interno).
 - **Hosting leggero** di più siti/vetrine su una VPS, ognuno isolato nella sua Docker.
 - **SSO** su più domini con provider aziendali (AD/OIDC).
 - **Reverse-proxy centralizzato** con regole d'accesso diverse per servizio.
-

Xal-Tor-Ka è software di SFS.it. Screenshot da installazione dimostrativa con dati di esempio.